Controllable generation with diffusion models. Methods for controllable generation using diffusion models have had significant impact, both in the context of image and video generation. For example, existing techniques allow controllable image generation conditioned on text, depth maps, edges, pose, or other signals (Zhang et al., 2023; Ye et al., 2023). While Chen et al. (2024) developed a ControlNet-based mechanism for transformer-based diffusion, limited to spatial conditioning with spatial signals, we explore conditioning mechanisms for camera poses (a spatio-temporal signal with an intricate temporal component) in a spatio-temporal transformer. Furthermore, there is a line of work for 3D generation that conditions diffusion models on camera poses for view-consistent multi-view generation (Watson et al., 2023; Tseng et al., 2023; Chan et al., 2023; Yu et al., 2023a; Kumari et al., 2024; Müller et al., 2024; Gao et al., 2024c). Our approach is most similar to related techniques in video generation that seek to control the camera position. For example, MotionCtrl (Wang et al., 2023e) designs camera and object control mechanisms for the VideoCrafter1 (Chen et al., 2023b) and SVD (Blattmann et al., 2023a) models. However, MotionCtrl is designed for U-Net-based approaches and does not directly apply to video diffusion transformers.

Concurrent 3D camera control methods. Concurrent approaches enable camera control by conditioning the temporal layers of the network with camera pose information, e.g., using Plücker coordinates (He et al., 2024a; Guo et al., 2024; Xu et al., 2024b; Kuang et al., 2024) or other embeddings (Yang et al., 2024b). Interestingly, it is also possible to incorporate camera control into video generation models without additional training through manipulation and masking of attention layers, though this requires additional tracking, segmentation, or depth for each input video (Hu et al., 2024; Xiao et al., 2024; Hou et al., 2024). Another recent work (Ling et al., 2024b) transfers motion, including camera motion, to other generated videos.

Although these approaches demonstrate promising results for U-Net-based video diffusion models, the techniques are not applicable to modern video transformers that model spatio-temporal dynamics jointly. While another concurrent work (Watson et al., 2024) uses a transformer-based architecture for space and time, it does not tackle text-based generation for dynamic scenes but focuses on novel view synthesis from an input image. In our work, we design an efficient mechanism that enables camera control in video diffusion transformers using a ControlNet inspired mechanism without sacrificing visual quality.

3 METHOD

3.1 LARGE TEXT-TO-VIDEO TRANSFORMERS

Text-to-video generation. Diffusion models have emerged as the dominant paradigm for large-scale video generation (Ho et al., 2022b;a; Brooks et al., 2024). The standard setup considers the conditional distribution $p(\boldsymbol{x}|\boldsymbol{y})$ of videos $\boldsymbol{x} \in \mathbb{R}^{F \times H \times W}$ (consisting of F frames of $H \times W$ resolution) given their text descriptions $\boldsymbol{y} \in \mathcal{Y}^L$, consisting of L (possibly padded) tokens from the alphabet $\mathcal{Y}$. Following (Karras et al., 2022), our video diffusion framework assumes a denoising model $D_{\boldsymbol{\theta}} : (\tilde{\boldsymbol{x}}; \boldsymbol{y}, \sigma) \mapsto \hat{\boldsymbol{x}}$ that predicts a clean video $\hat{\boldsymbol{x}}$ from the corresponding noised input $\tilde{\boldsymbol{x}} = \boldsymbol{x} + \sigma\boldsymbol{\varepsilon}$, where $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ is standard Gaussian noise and $\sigma \sim \log\mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$ is the noise strength, sampled from log-normal distribution with location P_{mean} and scale P_{std}. The model is parametrized by a neural network $F_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}; \boldsymbol{y}, \sigma)$ as $D_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}; \boldsymbol{y}, \sigma) = c_{\text{out}}(\sigma) F_{\boldsymbol{\theta}}(c_{\text{in}}(\sigma)\tilde{\boldsymbol{x}}; \boldsymbol{y}, \sigma) + c_{\text{skip}}(\sigma)\tilde{\boldsymbol{x}}$, where $c_{\text{in}}(\sigma), c_{\text{out}}(\sigma)$ and $c_{\text{skip}}(\sigma)$ are input, output and residual scaling factors from (Karras et al., 2022). The minimization objective is defined as

$$\mathcal{L}(\boldsymbol{\theta}) = \mathop{\mathbb{E}}_{p(\boldsymbol{x}, \boldsymbol{y}, \sigma, \boldsymbol{\varepsilon})} \left[\| D_{\boldsymbol{\theta}}(\boldsymbol{x} + \sigma\boldsymbol{\varepsilon}; \boldsymbol{y}, \sigma) - \boldsymbol{x} \|_2^2 \right]. \tag{1}$$

We refer the reader to (Menapace et al., 2024) and (Karras et al., 2022) for further details on the diffusion setup, which we adopted without modifications.

Spatiotemporal transformers. Following SnapVideo (Menapace et al., 2024), our video generator consists of two models: the base 4B-parameters generator, operating on 16-frames 36×64 resolution videos, and a 288×512 upsampler. The latter, a diffusion model itself, is fine-tuned from the base model and conditioned on the generated low-resolution videos. Each model uses FIT transformer blocks (Chen & Li, 2023; Jabri et al., 2022) for efficient self-attention operations (see Fig. 3). An FIT model consists of B blocks (we have $B = 6$ in all the experiments) and first partitions each frame in an input video into patches (Dosovitskiy et al., 2021) of resolution $h_p \times w_p$ (we use

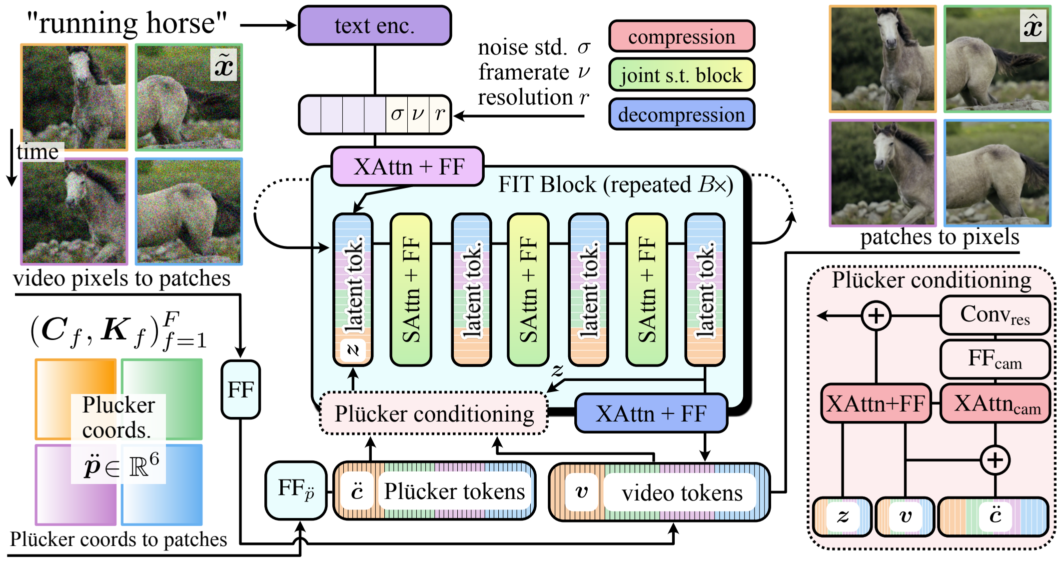

Figure 3: **Overview of architecture.** We adapt a FIT-based architecture (Chen & Li, 2023) to incorporate camera control. We take as input the noisy input video $\tilde{\boldsymbol{x}}$, camera extrinsics $\boldsymbol{C}_f$, and camera intrinsics $\boldsymbol{K}_f$ for each video frame f. We compute the Plücker coordinates for each pixel within the video frames using the camera parameters. Both the input video and Plücker coordinate frames are converted to patch tokens, and we condition the video patch tokens using a mechanism similar to ControlNet (Zhang et al., 2023) ("Plücker conditioning" block). Then, the model estimates the denoised video $\hat{\boldsymbol{x}}$ by recurrent application of FIT blocks (Chen & Li, 2023). Each block reads information from the patch tokens into a small set of latent tokens on which computation is performed. The results are written to the patch tokens in an iterative denoising diffusion process.

$h_p = w_p = 4$ in all the experiments). These video patches are then independently projected via a feedforward (FF) layer to obtain a sequence of video tokens $\boldsymbol{v}_{\ell:L}^{(b)} \triangleq (\boldsymbol{v}_1, ..., \boldsymbol{v}_L) \in \mathbb{R}^{L\times d}$ of length $L = F \times (H/h_p) \times (W/w_p)$ and dimensionality d. Next, each FIT block "reads" the information from this video sequence into a much shorter sequence of M *latent* tokens $\boldsymbol{z}_{m:M}^{(b)} \triangleq (\boldsymbol{z}_1, ..., \boldsymbol{z}_M)$ through a "`read`" cross-attention layer, followed by a feedforward layer. The core processing with self-attention layers is performed in this latent space, and then the result is written back to the video tokens through a corresponding "`write`" cross-attention layer (also followed by an FF layer). The latent tokens in each subsequent FIT block are initialized from the previous one, which helps to propagate the computational results throughout the network. In this way, the entire computation occurs jointly in both spatial and temporal axes, which yields superior scalability (Menapace et al., 2024). However, it abandons the decomposed spatial/temporal computation nature of modern video diffusion U-Nets, which modern camera conditioning techniques (Wang et al., 2023e; Yang et al., 2024b) rely on to enforce control without compromising visual quality.

3.2 Camera control for spatiotemporal transformers

Spatiotemporal camera representation. The standard way of representing camera parameters for a video (in the pinhole model) is via a trajectory of extrinsics and intrinsics camera parameters $(\boldsymbol{C}_f, \boldsymbol{K}_f)_{f=1}^F$ for each f-th frame. The matrix $\boldsymbol{C}_f = [\boldsymbol{R}; \boldsymbol{t}] \in \mathbb{R}^{3\times 4}$, describes the camera rotation $\boldsymbol{R} \in \mathbb{R}^{3\times 3}$ and translation $\boldsymbol{t} \times \mathbb{R}^3$, and $\boldsymbol{K}_f \in \mathbb{R}^{3\times 3}$ contains the focal length and principal point (and also horizontal/vertical skew coefficient, but it is always 0 in our setup). To control camera motion, existing methods condition the temporal attention layers of U-Net-based video generators on embeddings computed from these camera parameters (Wang et al., 2023e; Yang et al., 2024b; Hu et al., 2024). Such a pipeline provides a good conditioning signal for convolutional video generators with decomposed spatial/temporal computation, but our experiments demonstrate that it works poorly for spatiotemporal transformers: they either fail to pick up any controllability (when being added as transformed residuals to the latent tokens), or degrade the visual quality of the output (when the original network parameters are being fine-tuned). This motivates us to design a better camera conditioning scheme, tailored for modern large-scale spatiotemporal transformers.

First, we propose to normalize the camera parameters w.r.t the first frame. For this, we recompute the rotations and translations for each f-th frame as $\boldsymbol{R}'_f = \boldsymbol{R}_1^{-1}\boldsymbol{R}_f$ and $\boldsymbol{t}'_f = \boldsymbol{t}_f - \boldsymbol{t}_1$. This procedure results in normalized camera extrinsics as $\boldsymbol{C}_f{}' = [\boldsymbol{R}'_f; \boldsymbol{t}'_f]$ and establishes a consistent coordinate system across different samples in the dataset. After that, we found it essential to enrich the conditioning information by switching from temporal frame-level camera parameters to pixel-wise *spatiotemporal* ones. This is achieved by computing the Plücker coordinates for each pixel, providing a fine-grained positional representation.

Plücker coordinates provide a convenient parametrization of lines in the 3D space, and we use them to compute fine-grained positional representations of each pixel in each frame of a video. Given the extrinsic and intrinsic camera parameters $\boldsymbol{R}'_f, \boldsymbol{t}'_f, \boldsymbol{K}_f$ of the f-th frame, we parametrize each (h, w)-th pixel as a Plücker embedding $\ddot{\boldsymbol{p}}_{f,h,w} \in \mathbb{R}^6$ from the camera position to the pixel's center as

$$\ddot{\boldsymbol{p}}_{f,h,w} = (\boldsymbol{t}_f{}' \times \hat{\boldsymbol{d}}_{f,h,w}, \hat{\boldsymbol{d}}_{f,h,w}), \quad \hat{\boldsymbol{d}}_{f,h,w} = \frac{\boldsymbol{d}}{\|\boldsymbol{d}_{f,h,w}\|}, \quad \boldsymbol{d}_{f,h,w} = \boldsymbol{R}_f{}'\boldsymbol{K}_f[w, h, 1]^\top + \boldsymbol{t}_f{}'. \quad (2)$$

This approach mirrors the technique used in recent 3D works (Sitzmann et al., 2021; Chen et al., 2023a; Kant et al., 2024) as well as CameraCtrl (He et al., 2024a), a concurrent study focusing on camera control in U-Net-based video diffusion models. The motivation for using Plücker coordinates is that geometric manipulations in the Plücker space can be performed through simple arithmetics on the coordinates, which makes it easier for the network to use the positional information stored in such a disentangled representation.

Computing Plücker coordinates for each pixel results in a $\ddot{\boldsymbol{P}} \in \mathbb{R}^{6\times F\times H\times W}$ spatiotemporal camera representation for a video. To input it into the model, we first perform the equivalent ViT-like (Dosovitskiy et al., 2021) $h_p \times w_p$ patchification procedure. It is followed by a learnable 2-layered MLP $\text{MLP}_{\ddot{p}}$ with a GELU (Hendrycks & Gimpel, 2016) non-linearity to obtain the Plücker camera tokens sequence $\ddot{\boldsymbol{c}}_{\ell:L} \in \mathbb{R}^{L\times d}$ of the same length $L = F \times (H/h_p) \times (W/w_p)$ and dimensionality d as the video tokens sequence $\boldsymbol{v}^{(b)}_{\ell:L}$. This spatiotemporal representation carries fine-grained positional information about each pixel in a video, making it easier for the generator to accurately follow the desired camera motion.

Camera conditioning. To input the Plücker embeddings into our video generator, we design an efficient ControlNet like (Zhang et al., 2023) mechanism tailored for large transformer models (see Fig. 3). This mechanism is guided by two main objectives: 1) the model should be amenable to rapid fine-tuning from a small dataset with estimated camera positions; and 2) the visual quality shouldn't be compromised during the fine-tuning stage. We found that meeting these objectives is more challenging for spatiotemporal transformers compared to U-Net-based models with decomposed spatial/temporal computation, since even minor interventions into their design quickly lead to degraded video outputs. We hypothesize that the core reason for it is the entangled spatial/temporal computation of video transformers: any attempt to alter the temporal dynamics (such as camera motion) influences spatial communication between the tokens, leading to unnecessary signal propagation and overfitting during the fine-tuning stage. To mitigate this, we input the camera information gradually through `read` cross-attention layers, zero-initialized from the original network parameters of the corresponding layers.

Specifically, in each b-th FIT block of our video generator, we replace its standard `read` cross-attention operation (see (Jabri et al., 2022; Menapace et al., 2024) for details):

$$\boldsymbol{z}'^{(b)}_{m:M} = \text{FF}^{(b)}(\text{XAttn}^{(b)}(\boldsymbol{z}^{(b)}_{m:M}, \boldsymbol{v}^{(b)}_{\ell:L})), \quad (3)$$

where $\text{FF}(\cdot)$ and $\text{XAttn}(\cdot, \cdot)$ denote feed-forward and cross-attention layers respectively. Our revised formulation is given as:

$$\boldsymbol{z}'^{(b)}_{m:M} = \text{FF}^{(b)}(\text{XAttn}^{(b)}(\boldsymbol{z}^{(b)}_{m:M}, \boldsymbol{v}^{(b)}_{\ell:L})) + \text{Conv}^{(b)}_{\text{res}}\left[\text{FF}^{(b)}_{\text{cam}}(\text{XAttn}^{(b)}_{\text{cam}}(\boldsymbol{z}^{(b)}_{m:M}, \ddot{\boldsymbol{c}}_{\ell:L} + \boldsymbol{v}^{(b)}_{\ell:L}))\right], \quad (4)$$

where $\text{FF}^{(b)}_{\text{cam}}$, and $\text{XAttn}^{(b)}_{\text{cam}}$ are learnable layers, $\text{Conv}^{(b)}_{\text{res}}$ is a 1-dimensional convolution that processes camera-augmented latents. The produced latents $\boldsymbol{z}'^{(b)}_{m:M}$ are then passed to the sequence of four